

Министерство науки и высшего образования Российской Федерации

Самарский национальный исследовательский университет
имени академика С.П. Королева

Трансформеры

Методические указания к лабораторной работе № 4
по курсу «НЕЙРОННЫЕ СЕТИ И ГЛУБОКОЕ ОБУЧЕНИЕ»

САМАРА

2026

Составители: д.ф.-м.н., профессор В.В.Мясников
к.т.н. А.Ю.Баврина

УДК 681.3

Трансформеры

Методические указания к лабораторной работе № 4
Самарский национальный исследовательский университет
имени академика С.П. Королева
Составители: В.В. Мясников, А.Ю.Баврина
Самара, 2026. 25 с.

В лабораторной работе № 4 «Трансформеры» дисциплины «Нейронные сети и глубокое обучение» изучаются принципы работы и основные функции построения, обучения, оптимизации и оценки качества работы нейронных сетей типа трансформеры.

Методические указания предназначены для студентов, обучающихся по специализациям 10.05.03 - "Информационная безопасность автоматизированных систем" и 01.03.02 - "Прикладная математика и информатика".

Печатается по решению редакционно-издательского совета
Самарского национального исследовательского университета
имени академика С.П. Королева

Рецензент: д.ф.-м.н., профессор ФИО

Цель работы: изучить принципы работы и основные функции построения, обучения, оптимизации и оценки качества работы нейронных сетей типа трансформеры.

ТРАНСФОРМЕРЫ

Трансформер (transformer) – нейросетевая архитектура, составленная из слоев, сочетающих механизм (само-)внимания, и полносвязные слои. Трансформер был предложен в работе в 2017 году коллективом авторов [1], многие из которых относились к исследовательским подразделениям компании Google (Google Brain, Google Research).

1 Общая структуры трансформера

Трансформер состоит из двух независимых, но похожих по структуре частей: *трансформер-кодер* (encoder) и *трансформер-декодер* (decoder), как показано на рисунке 1. *Трансформер-кодер* (на рисунке слева) преобразует последовательность (в общем случае – набор/множество) из n входных векторов $(x_1^{in}, x_2^{in}, \dots, x_n^{in})$, каждый из которых имеет размерность d , в последовательность $(e_1, e_2, \dots, e_n)$ элементов той же размерности d . В свою очередь, *трансформер-декодер* (на рисунке справа) использует последовательность $(e_1, e_2, \dots, e_n)$, полученную из кодера, для последовательного (поэлементного) синтеза последовательности-результата $(y_1, y_2, \dots, y_m)$ некоторой, заранее неопределенной, длины m .

Ниже будут разобраны основные элементы этой архитектуры:

- «Scaled Dot-Product Attention» - раздел 2;
- «Multi-Head Attention» - раздел 3;
- «Positional Encoding» и «Input Embedding» - 4;
- «Feed Forward» - подраздел 5;

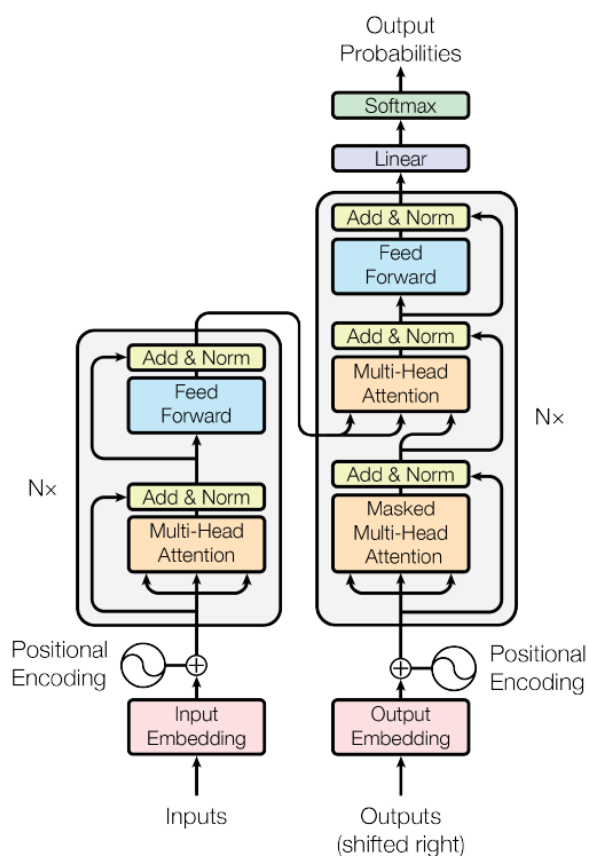
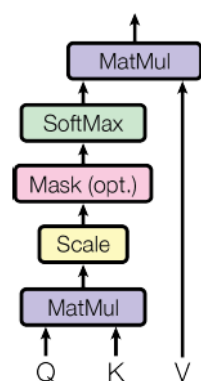


Рисунок 1 – Архитектура модели «трансформер» (англ.: Transformer),
по материалам работы [1]

Scaled Dot-Product Attention



Multi-Head Attention

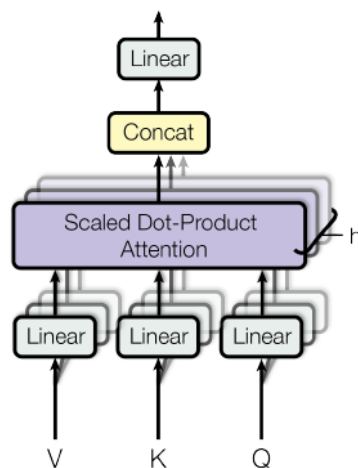


Рисунок 2 – Механизм SDPA (слева) и многоголового внимания МНА (справа)

2 Scaled Dot-Product Attention (SDPA)

Пусть K – матрица размером $n \times d$, строки которой (вектора k_i^T) мы будем интерпретировать как «ключи» (англ.: Keys), V – матрица размером $n \times d_v$, строки которой (вектора v_i^T) мы будем интерпретировать как «значения» (англ.: Values). Очевидно, величина n – это число пар ключ-значение. И пусть задан вектор q

размером d , который мы будем интерпретировать как «запрос» (англ.: Queries). Обычный (маскированный SDPA - «masked SDPA» - рассмотрен ниже) блок внимания SDPA задается в виде выпуклой комбинации векторов-значений следующим образом (результат выражения – вектор-строка):

$$SDPA(q, K, V) \triangleq \left(SoftMax \left(\frac{Kq}{\sqrt{d}} \right) \right)^T V = \sum_{i=1}^n SoftMax_i \left(\frac{Kq}{\sqrt{d}} \right) v_i^T, \quad (1)$$

здесь $SoftMax_i$ – взятие i -ого компонента вектора-результата функции SoftMax.

Функция softmax обладает всеми требуемыми свойствами:

- дифференцируемость по аргументу(ам),
- неотрицательность значений,
- нормированность.

Недостатком SDPA является отсутствие в нем параметров.

Если задан не один, а m запросов, сформированных в матрицу Q размером $m \times d$ (строки матрицы – суть вектора-запросы q_i^T), будем интерпретировать запись $SDPA(Q, K, V)$ как матрицу размером $m \times d_V$ следующего вида:

$$SDPA(Q, K, V) \triangleq \begin{bmatrix} SDPA(q_1, K, V) \\ \vdots \\ SDPA(q_m, K, V) \end{bmatrix} \quad (2)$$

Свойства SDPA

Пусть π - некоторая заданная перестановка строк матрицы. Тогда справедливы:

$$\begin{aligned} SDPA(Q, \pi(K), \pi(V)) &= SDPA(Q, K, V), \\ SDPA(\pi(Q), K, V) &= \pi(SDPA(Q, K, V)). \end{aligned} \quad (4)$$

Пусть $W^V \in \mathbb{R}^{d_V \times d_*}$, а $W \in \mathbb{R}^{d \times d}$ –квадратная матрица полного ранга, тогда справедливы:

$$\begin{aligned} SDPA(Q, K, VW^V) &= SDPA(Q, K, V)W^V, \\ SDPA(QW, KW, V) &= SDPA(Q, K, V). \end{aligned} \quad (5)$$

3 Многоголовое внимание

3.1 Нативный блок многоголового внимания

Многоголовое внимание (англ.: МНА – multi-head attention) - преобразование

триплета матриц запросов Q , ключей K и значений V с использованием h (в оригинале - число «голов», англ.: head) отображений указанных матриц в пространства меньшей размерности, последующего применения к получаемым триплетам механизма SDPA и объединением результатов с использованием линейного преобразования. Формально:

$$\begin{aligned} MHA(Q, K, V) &= \left(SDPA(QW_1^Q, KW_1^K, VW_1^V), \dots, SDPA(QW_h^Q, KW_h^K, VW_h^V) \right)^T W^o = (\\ &= \sum_{i=1}^h SDPA(QW_i^Q, KW_i^K, VW_i^V) W_i^o . \end{aligned}$$

6)

В представленном выражении:

$$W^o = \begin{pmatrix} W_1^o \\ W_2^o \\ \vdots \\ W_h^o \end{pmatrix}.$$

Размерности используемых матриц:

$$W_i^Q \in \mathbb{R}^{d \times d_K}, W_i^K \in \mathbb{R}^{d \times d_K}, W_i^V \in \mathbb{R}^{d \times d_V}, W^o \in \mathbb{R}^{hd_V \times d}, W_i^o \in \mathbb{R}^{d_V \times d}, i = \overline{1, h},$$

(7)

$$d_K h = d_V h = d. \quad (8)$$

С учетом указанных размеров матриц имеем размерность результата МНА:

$$MHA(Q, K, V) \in \mathbb{R}^{m \times d}.$$

В оригинальной работе: $h=8, d = 512, d_K = d_V = 64$.

Свойства МНА:

- возможность настраиваться в процессе обучения за счет наличия параметров (7);
- возможность настраиваться в процессе обучения на различные «аспекты» или «контексты» данных за счет использования нескольких параметризованных SDPA (нескольких «голов»);
- устраняется проблема с «самовниманием»;
- . - число обучаемых параметров не зависит от числа голов.

Формальные свойства:

$$\begin{aligned} MHA(Q, \pi(K), \pi(V)) &= MHA(Q, K, V), \\ MHA(\pi(Q), K, V) &= \pi(MHA(Q, K, V)). \end{aligned} \quad (11)$$

$$MHA(QW, KW, V) = MHA(Q, K, V). \quad (12)$$

3.3 Маскированное многоголовое внимание

Маскированное многоголовое внимание (Masked MHA) используется в декодере. Суть маскирования – исключить из процесса вычислений те данные, которые на текущий момент доступными не являются. Для этого блок SDPA модифицируется следующим образом (ниже вектор $\delta \in \{0,1\}^n$ определяет маску, значения «1» в нем означают использование соответствующего элемента последовательности):

$$MSDPA(q, K, V) \triangleq \left(SoftMax \left(\frac{Kq}{\sqrt{d}} \circ \delta + \langle -\infty \rangle (\bar{1} - \delta) \right) \right)^T V \quad (13)$$

С использованием введенного маскированного SDPA вводится и блок *маскированного многоголового внимания*:

$$\begin{aligned} MMHA(Q, K, V) &= \left(MSDPA(QW_1^Q, KW_1^K, VW_1^V), \dots, MSDPA(QW_h^Q, KW_h^K, VW_h^V) \right)^T W^O = \\ &= \sum_{i=1}^h MSDPA(QW_i^Q, KW_i^K, VW_i^V)^T W_i^O. \end{aligned} \quad (13^*)$$

4 Кодирование поступающей информации

4.1 Входные векторы (Input embeddings)

В общем случае вид входных векторов x_j^{in} ($j = \overline{0, n-1}$), обозначаемых на оригинальном рисунке трансформера как «Inputs», определяется конкретной прикладной задачей. Для рассматриваемого трансформера 2017 года j -ый входной вектор x_j^{in} определяется следующим образом:

$$x_j^{in} = W^{in} b_j^{in}, \quad (4a)$$

Здесь $b_j^{in} = (0 \ \dots \ 0 \ 1 \ 0 \ \dots \ 0)^T$ – j -ый входной *индикаторный вектор* (англ.: one-hot vector) размерности $vocabIn$, равной числу возможных токенов/слов в словаре, а $W^{in} \in \mathbb{R}^{d \times vocabIn}$ – матрица, столбец которой – суть вектор представления (*эмбединг*) конкретного токена/слова входного текста.

4.2 Позиционное кодирование

Для того, чтобы трансформер мог учитывать информацию о расположении обрабатываемых векторов (в последовательности или на изображении), в качестве входных векторов x_i в него передаются суммы исходных векторов x_i^{in} и позиционных кодов p_i , то есть векторов, которые содержат информацию о расположении вектора:

$$x_i = x_i^{in} + p_i, \quad i = \overline{1, n}. \quad (13)$$

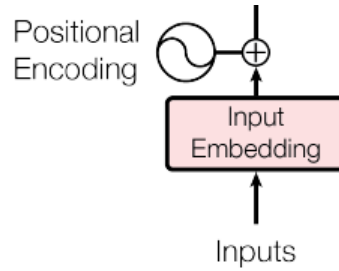


Рисунок 3. Из статьи [1]

Соответствующие компоненты позиционных векторов задаются в виде (i пробегает по индексам векторов, а величина j – по их размерности):

$$p_{i,2j} = \sin\left(\frac{i}{R^{2j/d}}\right), \quad p_{i,2j+1} = \cos\left(\frac{i}{R^{2j/d}}\right), \quad i = \overline{1, n}; j = \overline{0, \frac{d}{2} - 1}. \quad (14)$$

Здесь $R=10\,000$ – максимальная длина последовательности. Для $d = 6$ мы имеем следующий 6-мерный позиционный вектор для i -го токена:

$$p_i = \left(\sin(i), \cos(i), \sin\left(\frac{i}{R^{1/3}}\right), \cos\left(\frac{i}{R^{1/3}}\right), \sin\left(\frac{i}{R^{2/3}}\right), \cos\left(\frac{i}{R^{2/3}}\right) \right)^T.$$

5 Позиционная сеть прямого распространения

Позиционная сеть прямого распространения (англ.: PositionWise FFN – feed-forward network) – обычная сеть прямого распространения, но использующая одни и те же параметры сети для всех векторов в наборе. Конкретно, если матрица $X \in \mathbb{R}^{n \times d}$ составлена из поступившего набора d -мерных векторов

$$X = \begin{pmatrix} x_1^T \\ x_2^T \\ \vdots \\ x_n^T \end{pmatrix},$$

то результатом является следующая матрица с n строками:

$$PW_FFN(X) \triangleq \begin{pmatrix} (FFN(x_1))^T \\ (FFN(x_2))^T \\ \vdots \\ (FFN(x_n))^T \end{pmatrix}. \quad (15^*)$$

В качестве FF-сети в трансформере используется двуслойная сеть, с активационной функцией ReLU на первом слое и с тождественной функцией на втором. В результате:

$$FFN(x) \triangleq W_2 ReLU(W_1 x + b_1) + b_2. \quad (16)$$

Здесь $W_1 \in \mathbb{R}^{d_{ff} \times d}$, $b_1 \in \mathbb{R}^{d_{ff}}$; $W_2 \in \mathbb{R}^{d \times d_{ff}}$, $b_2 \in \mathbb{R}^d$, а $d_{ff} = 2048 (= 4d)$.

6 Трансформер-кодер

6.1 Трансформер-кодер: архитектура основного блока

Архитектура основного блока кодера трансформера представлена на рисунке 3.

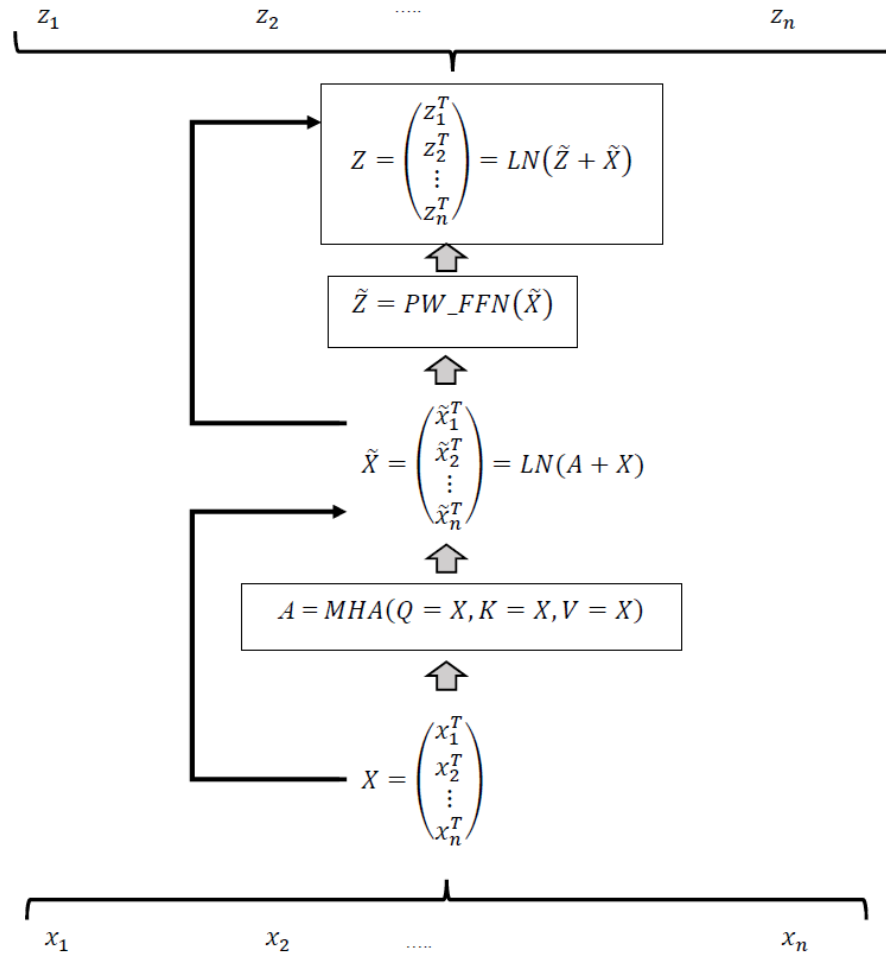


Рисунок 3 - Архитектура основного блока кодера трансформера (LN – Layer Norm)

В виде псевдокода блок кодера может быть представлен следующим образом:

```

def EncoderBlockPAR(X):
     $\tilde{X} = LN_{\gamma_1, \beta_1}(MHA(Q = X, K = X, V = X) + X)$ 
     $Z = LN_{\gamma_2, \beta_2}(PW\_FFN(\tilde{X}) + \tilde{X})$ 
    return Z

```

В блоке кодера использованы два ключевых момента, существенным образом влияющих на его эффективность:

- параметризованный механизм многоголового внимания;
- использование skip-соединений, предложенных в СНС ResNet.

Число параметров блока кодера трансформера приведено в таблице 1.

Таблица 1 Состав и число параметров основного блока трансформера-кодера

Этап	Количество параметров	
	выражение	Численно
МНА	$4d^2$	2^{20}
PW_FFN	$2dd_{ff} + d + d_{ff}$	$2^{21} + 2^{11} + 2^9$
LN, 1 и 2	4	2^2
ИТОГО	$4d^2 + 2dd_{ff} + d + d_{ff} + 4$	$2^{21} + 2^{20} + 2^{11} + 2^9 + 2^2 = 3\,148\,292$

6.2 Трасформер-кодер: общая архитектура

Реализация трансформера-кодера в виде псевдокода представлена ниже.

```

def Encoder( $X^{in}$ ,  $N = 6$ ):
     $E = X^{in} + P$ 
    for k in range(N):
         $E = \text{EncoderBlock}_{PAR(k)}(E)$ 
    return E

```

Здесь $PAR(k)$ – набор параметров для k-го блока трансформера-кодера.

Учитывая, что число блоков кодера для базового варианта трансформера $N=6$, имеем следующее выражение для числа параметров:

$$N(4d^2 + 2dd_{ff} + d + d_{ff} + 4) + (vocabIn + 1)d$$

Что при $N = 6$ блоках и размере словаря $vocabIn = 25000$ дает 31 690 264 обучаемых параметров.

7 Трасформер-декодер

В то время как на кодер трансформера поступает целиком вся входная последовательность $(x_1^{in}, x_2^{in}, \dots, x_n^{in})$, в результате чего кодер выдает последовательность $(e_1, e_2, \dots, e_n)$ той же длины (формирующая матрицу E), синтез

выходной последовательности $(y_1, y_2, \dots, y_m)$ происходит поэлементно и последовательно, то есть декодер используется каждый раз, когда необходимо получить следующий элемент последовательности. Синтез завершается, когда в качестве следующего токена/слова предсказывается токен конца последовательности [EOS].

7.1 Трасформер-декодер: архитектура основного блока

Архитектура основного блока декодера представлена на рисунке 4.

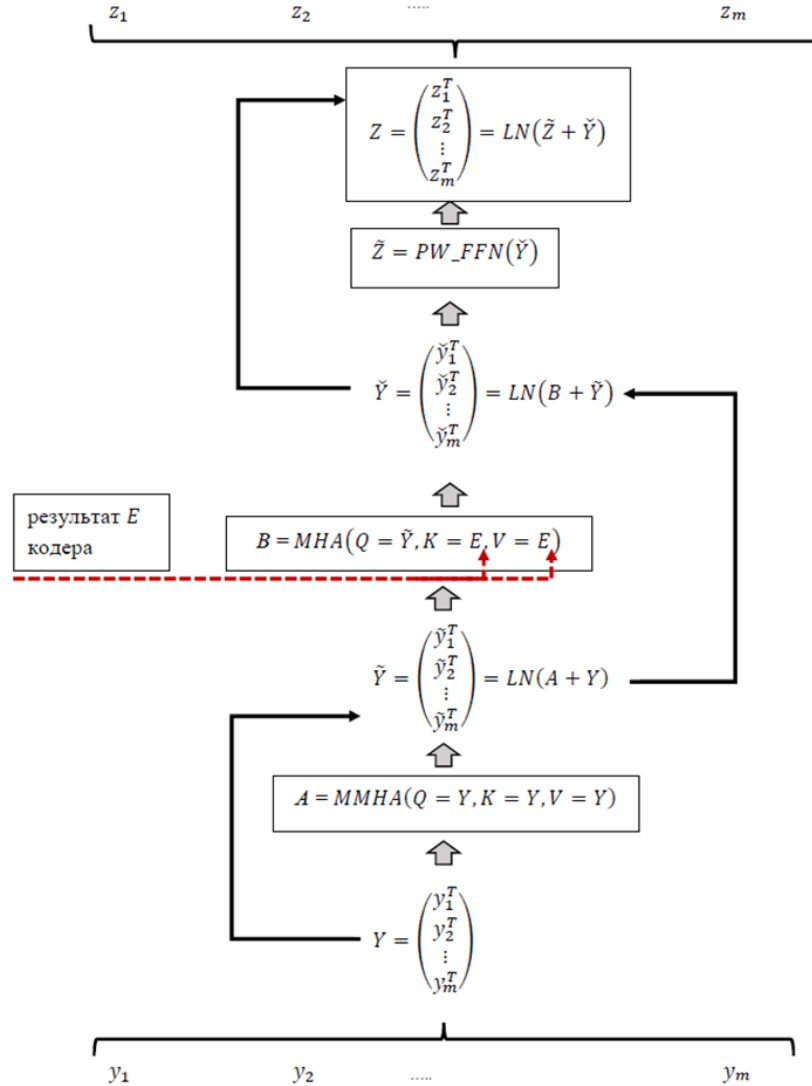


Рисунок 4 - Архитектура основного блока декодера трансформера

В виде псевдокода блок декодера может быть представлен следующим образом (PAR – все параметры соответствующего блока декодера):

```
def DecoderBlockPAR(Y, E):
     $\tilde{Y} = LN_{\tilde{\gamma}_1, \tilde{\beta}_1}(MMHA(Q = Y, K = Y, V = Y) + Y)$ 
     $\check{Y} = LN_{\check{\gamma}_2, \check{\beta}_2}(MHA(Q = \tilde{Y}, K = E, V = E) + \tilde{Y})$ 
     $Z = LN_{\gamma_2, \beta_2}(PW\_FFN(\check{Y}) + \check{Y})$ 
```

return Z

Число обучаемых параметров для одного основного блока трансформера-декодера представлено в таблице 2.

Таблица 2 Состав и число параметров основного блока трансформера-декодера

Этап	Количество параметров	
	выражение	Численно
ММНА	$4d^2$	2^{20}
МНА	$4d^2$	2^{20}
PW_FFN	$2dd_{ff} + d + d_{ff}$	$2^{21} + 2^{11} + 2^9$
LN, 1-3	$3 \cdot 2$	4
ИТОГО	$8d^2 + 2dd_{ff} + d + d_{ff} + 6$	$2^{22} + 2^{11} + 2^9 + 6 = 4\,196\,870$

7.2 Трасформер-декодер: общая архитектура

Входными данными декодера, на рисунке названными «outputs», является последовательность $(y_1, y_2, \dots, y_m)$ векторов-эмбедингов уже предсказанных токенов/слов. Выходными данными декодера является последовательность $(g_1, g_2, \dots, g_m)$ векторов вероятностей появления конкретного токена в качестве следующего предсказываемого элемента выходной последовательности (на рисунке названные «outputs probabilities»). Каждый вектор g с вероятностями имеет размерность $vocabOut$ по числу слов/токенов на языке получаемого текста. Связь между выходом и входом декодера может быть представлена следующим образом:

$$y_{m+1} = W^{out} b_{m+1}^{out}, \quad b_{m+1}^{out} = BinMax(q_m).$$

Функция $BinMax$ формирует элементы вектора b_{m+1}^{out} из вектора g_m в соответствии с выражением:

$$BinMax: \quad b_{m+1,j}^{out} = \begin{cases} 0, & j \neq \operatorname{argmax} g_m; \\ 1, & j = \operatorname{argmax} g_m; \end{cases} \quad j = \overline{1, vocabOut}. \quad (19)$$

Вектор b_{m+1}^{out} – предсказанный индикаторный вектор для «следующего» $(m+1)$ -го токена генерируемой последовательности; W^{out} – матрица $\in \mathbb{R}^{vocabOut \times d}$, строки которой – суть эмбединги представления соответствующих токенов языка выходного текста.

Основной блок декодера, как видно из рисунка, используется в декодере последовательно N раз (в работе $N=6$), при этом предварительно вектора-эмбединги

входной последовательности преобразуются с использованием позиционного кодирования.

Реализация трансформера-декодера в виде псевдокода представлена ниже.

```
def Decoder(Y, E, N=6):
    Z=Y+ P
    for k in range(N):
        Z = DecoderBlockPAR(k)(Z, E)
    (g1, g2, ..., gm): gi = softmax(WdecodeZiT(6) + bdecode), i =  $\overline{1, n}$ .
    return (g1, g2, ..., gm)
```

Здесь P – та же матрица позиционного кодирования, что и в подразделе 4.2.

Матрица W^{decode} и вектор b^{decode} , также как матрицы W^{in} и W^{out} , настраиваются в режиме обучения трансформера.

Числа обучаемых параметров декодера (число блоков N=6):

$$N(8d^2 + 2dd_{ff} + d + d_{ff} + 6) + (vocabOut + 1)d,$$

что в численном выражении при $vocabOut=vocabIn=25000$ дает величину 37 981 732. Результат – во всем трансформере порядка 68 млн параметров.

8 Режимы использования трансформера

8.1 Режим применения (обученного) трансформера (инференс)

Подготовительные действия:

- преобразование входной последовательности в последовательность токенов,
- финализация (завершение, добавление в конец) входной последовательности токенов с использованием [CLS],
- преобразование последовательности токенов в последовательность индикаторных векторов

$$(b_1^{in}, b_2^{in}, \dots, b_n^{in});$$

- последовательность индикаторных векторов преобразуется в последовательность эмбедингов $(x_1^{in}, x_2^{in}, \dots, x_n^{in})$ с использованием преобразования (4a), а именно:

$$x_j^{in} = W^{in}b_j^{in}, \quad j = \overline{1, n}.$$

Применение кодера

Набор векторов-эмбедингов $(x_1^{in}, x_2^{in}, \dots, x_n^{in})$, сформированных в матрицу X^{in} , поступает на вход трансформера кодера:

$$E = \text{Encoder}(X^{in}, K).$$

Результатом является матрица, строки которой – векторы $(e_1, e_2, \dots, e_n)$.

Применение декодера

В виде псевдокода ($b_{[EOS]}$ - индикаторный вектор, соответствующий токenu [EOS]; $b_{[SOS]}$ – соответствующий токenu [SOS]; ниже также будем использовать подобное обозначение) процесс применения декодера может быть представлен в следующем виде:

$$\begin{aligned} & m=1 \\ & b_m^{out} = b_{[SOS]} \\ & \text{while } (b_m^{out} \neq b_{[EOS]}): \\ & \quad y_m = W^{out} b_m^{out} \\ & \quad (q_1, q_2, \dots, q_m) = \text{Decoder} \left(\begin{pmatrix} y_1^T \\ y_2^T \\ \vdots \\ y_m^T \end{pmatrix}, E \right) \\ & \quad (20) \\ & \quad b_{m+1}^{out} = \text{BinMax}(q_m) \\ & \quad m=m+1 \end{aligned}$$

Результатом применения декодера является последовательность индикаторных векторов $(b_1^{out}, b_2^{out}, \dots, b_m^{out})$, причем

$$b_1^{out} = b_{[SOS]} \quad b_m^{out} = b_{[EOS]}. \quad (20^*)$$

В режиме применения трансформера (инференс) запуск декодера происходит столько раз, какова длинна получаемой последовательности токенов!

Постобработка

Полученную последовательность индикаторных векторов $(b_1^{out}, b_2^{out}, \dots, b_m^{out})$ заменяют на последовательность токенов/слов, игнорируя первый и последний индикаторные вектора (20*).

8.2 Режим обучения трансформера

Пусть $(b_1^{in}, b_2^{in}, \dots, b_n^{in})$ - входная последовательность индикаторных векторов, а

$(b_1^{outGT}, b_2^{outGT}, \dots, b_m^{outGT})$ – верная последовательность индикаторных векторов для декодера, удовлетворяющая (20*). При обучении используется ВСЯ последовательность для декодера в правой части выражения (20), а определение ошибки осуществляется путем определения функции потерь между истинными значениями выходных индикаторных векторов и получаемыми для данного входа распределениями вероятностей на выходе декодера:

$$Loss(q_t \sim b_{t+1}^{outGT}, t = \overline{1, m-1}).$$

9 Некоторые модификации

9.1 Post-LN и Pre-LN

В частности, первоначальная архитектура, предложенная в оригинальной работе по трансформерам [1], предполагала использование LN после операций преобразования данных в блоке внимания или блоке FFN и агрегации их с исходными данными. Такой вариант построения трансформера, называемый *Post-LN Transformer* и изображенный на рисунке 5.a из статьи [2], может быть представлен в виде следующего выражения:

$$x^* = LN(x + F(x; \Theta)), F = \text{MHA} \text{ или } F = \text{FFN}. \quad (21)$$

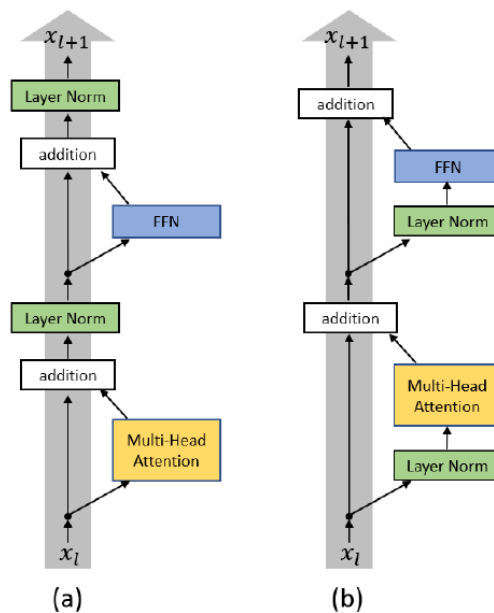


Рисунок 5 – Post-LN трансформер (a) и Pre-LN трансформер (b)

В качестве альтернативы к нему был предложен т.н. *Pre-LN Transformer*, показанный на рисунке 5.b. Реализуемое им преобразование может быть

представлено в виде:

$$x^* = x + F(LN(x); \Theta). \quad (22)$$

9.2 Модель BERT

Модель BERT [3] - Bidirectional Encoder Representations from Transformers – первая языковая модель, которая для решения ряда задач NLP (англ.: natural language processing) использовала архитектуру трансформера-кодера. Основным аргументом авторов для использования именно кодера вместо декодера являлась необходимость учета *двунаправленных связей* (внимания) между токенами/словами в предложении/контексте для его лучшего понимания. Такой подход имел существенное отличие от подхода в модели GPT (да и в классических однонаправленных РНС), где использовался декодер, а механизм внимания для конкретного токена учитывал только его предисторию.

Замечательным следствием принятого авторами решения об использовании именно кодера оказался и ставший возможным новый способ *предварительного обучения*. В отличие от GPT и РНС, в которых можно предсказывать распределение вероятностей следующего токена и обучать, максимизируя вероятность искомого, в модели BERT авторы применили два относительно новых способа. Первый из них, предполагал *случайное «маскирование» отдельных токенов* и обозначался авторами как *MLM* (англ.: “*masked language model*”). Второй предполагал решение задачи *предсказания/узнавания следующего предложения* - *NSP* (англ.: *Next Sentence Prediction*). Эти два нововведения позволили *BERT не только продемонстрировать значительное превосходство над другими существующими на тот момент моделями для NLP (включая GPT), но и инициировать прогресс в использовании трансформеров в области компьютерного зрения*. Иллюстрация к процессам обучения (предварительного и настройки) трансформера BERT дана на рисунке 6.

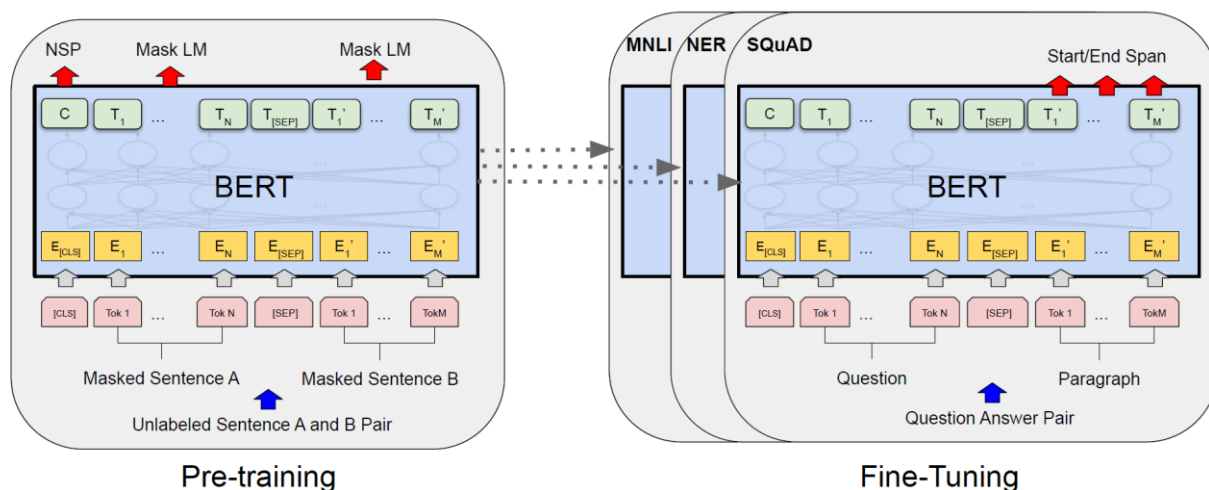


Figure 1: Overall pre-training and fine-tuning procedures for BERT. Apart from output layers, the same architectures are used in both pre-training and fine-tuning. The same pre-trained model parameters are used to initialize models for different down-stream tasks. During fine-tuning, all parameters are fine-tuned. [CLS] is a special symbol added in front of every input example, and [SEP] is a special separator token (e.g. separating questions/answers).

Рисунок 6 – Модель трансформера-кодера BERT

9.3 Модель GPT

Модель GPT или GPT-1 [4] – это первая модель типа GPT в плеяде известных моделей GPT-1/2/2.5/3/4 и других известной американской компании OpenAI. Эта модель, в отличие от оригинальной архитектуры трансформера, включающей кодер и декодер, использовала для решения задач NLP исключительно *декодер*.

Хотя серьезных архитектурных изменений в конструкции и использовании самого трансформера-декодера в работе не представлено (устранение кодера привело только к отказу от блоков MHA и нормализации в декодере, результирующий блок декодера приведен на рисунке 1 из статьи ниже), одним из основных вкладов рассматриваемой публикации, повлиявшим на дальнейшее развитие трансформеров и область NLP в целом, является введение концепции генеративного предварительного обучения (Generative Pre-Training). В рамках концепции генеративного предварительного обучения авторы предложили двухэтапный подход к обучению моделей NLP, включающий *предварительное обучение* (англ.: pre-training) и последующую *тонкую настройку* (англ.: fine-tuning). В процессе *предварительного обучения* модель обучается на большом объеме *неразмеченных текстовых данных* с использованием задачи *языкового моделирования* (англ.: language modeling): предсказания следующего слова в последовательности на основе предыдущих слов. Здесь следует еще раз отметить,

что для постановки задачи предсказания следующего слова в последовательности слов не требуется какой-либо подготовки данных ни вручную, ни автоматически (разметки, классификации, сортировки и др.), то есть для постановки такой задачи и обучения модели не требуется ничего, кроме самого корпуса текстов. При этом в процессе предварительного обучения модель сама адаптируется к языку текста, его стилистике, семантике и другим его особенностям. В результате, подход с предварительным обучением, фактически *самообучением*, позволил эффективно использовать огромные объёмы неразмеченных текстовых данных для улучшения понимания языка и повлиял на выбор названия полученного решения – генеративный предварительно обученный трансформер - GPT. Здесь термин «генеративный» указывает на способность модели генерировать (создавать) новый контент (например, текст), а «предварительно обученный» указывает на обязательное наличие этапа предварительного обучения (самообучения).

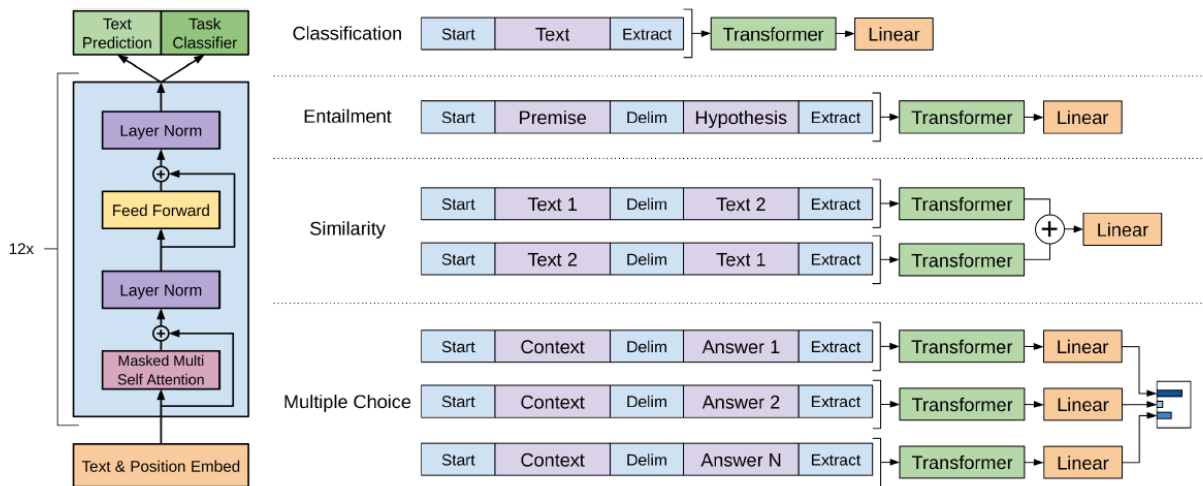


Figure 1: **(left)** Transformer architecture and training objectives used in this work. **(right)** Input transformations for fine-tuning on different tasks. We convert all structured inputs into token sequences to be processed by our pre-trained model, followed by a linear+softmax layer.

Рисунок 7 – Модель GPT

Для оценки качества синтезируемого текста, наряду со сравнением, часто используют перплексию (англ.: perplexity, PPL). *Перплексия*—мера того, насколько хорошо распределение вероятностей предсказывает выборку. Пусть построенная ИНС $F(\dots; W)$ с вектором параметров W используется для синтеза речи, то есть некоторой последовательности токенов/слов. Обозначим $y_0 y_1 y_2 y_3 \dots y_{T-1}$ – синтезированную ею последовательность токенов. И пусть

$$\hat{p}(y_t | y_0 \dots y_{t-1}) = F(y_0 \dots y_{t-2}; y_{t-1}; W) \quad (23)$$

Оцениваемая ИНС вероятность появления y_t при входной последовательности $y_0 \dots y_{t-1}$. Из определения условных вероятностей имеем:

$$p(y_0 \dots y_{T-1}) = p(y_0)p(y_1|y_0)p(y_2|y_0y_1) \dots p(y_{T-1}|y_0 \dots y_{T-2}).$$

Заменяя теоретические условные вероятности в последнем выражении их оценками из (7), а также «нормируя» произведение (по отношению к количеству множителей корнем соответствующей степени), выражение для перплексии можно задать в виде:

$$PP(y_0 \dots y_{T-1}) \equiv \frac{1}{\sqrt[T]{\prod_{t=0}^{T-1} F(y_0 \dots y_{t-1}; y_t; W)}}. \quad (24)$$

2. ЛИТЕРАТУРА

1. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., and Polosukhin, I. Attention is all you need. In Advances in Neural Information Processing Systems, 2017
2. Ruibin Xiongy, etc. On Layer Normalization in the Transformer Architecture // arXiv:2002.04745v2 [cs.LG] 29 Jun 2020
3. Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. In Proceedings of HLT-NAACL. Minneapolis, Minnesota, 4171–4186. <https://doi.org/10.18653/v1/N19-1423>
4. Alec Radford, Karthik Narasimhan, Tim Salimans, and Ilya Sutskever. 2018. Improving language understanding by generative pre-training. (2018).
5. Гудфеллоу Я., Бенджио И., А.Курвиль Глубокое обучение, ДМК Пресс, 2017
6. Бишоп К. М., Бишоп Х. Глубокое обучение: принципы и концепции, ДМК Пресс, 2025.
7. Принс Саймон Машинное обучение. От основ до продвинутых моделей (серия - Библиотека MIT), 2025.

3. ПОРЯДОК ВЫПОЛНЕНИЯ ЛАБОРАТОРНОЙ РАБОТЫ

3.1. Исходные данные

- набор данных en_fr.csv;
- код программы, файлы: Transformer.py, transformer_notebook_helper_module.py и данные/файлы о необходимом окружении.

3.2. Общий план выполнения работы¹

1. Загрузить набор данных для обучения и тестирования в соответствии с выданным вариантом.

2. Изучить данный код программы, разобраться в соответствии элементов трансформера, представленных в настоящих методических указаниях, и элементов программы.

Примечание: комментарии по коду доступны на сайте: <https://www.kaggle.com/code/henrychibueze/understanding-the-transformer-neural-network>

3. Обучить модель на обучающей выборке en_fr.csv (занимает порядка 15-20 мин, n_workers=0). Вывести (и сохранить в PDF) данные обучения для отчета (зависимость показателей качества от эпохи).

4. Сохранить текущую версию файла Transformer и создать несколько новых для проведения исследования качества работы трансформера от параметров на модели en_fr.csv. Исследуемые варианты (ограничиться обучением на 30000 примеров и длиной вектора 256):

- а) влияние замены функции активаций в блоке FFN с ReLU на GeLU;
- б) влияние расположения блока нормализации слоя LN - рассмотреть варианты Post_LN и Pre_LN (и для кодера, и для декодера), представленные в подразделе 9.1.
- в) влияние уменьшения числа нейронов на втором слое FFN в 2 и 4 раза;
- г) влияние изменения числа голов в МНА.

Сохранить графики зависимости показателей качества от эпохи и сделать выводы по результатам исследования.

¹ Рекомендуемая среда исполнения Python 3.10 с последовательно установленными модулями: pytorch (torch), pandas, scikit-learn (sklearn), matplotlib, tqdm, pyarrow.

5. Сдать результаты работы и отчет и ответить на вопросы.

4. КОНТРОЛЬНЫЕ ВОПРОСЫ

1. Что представляет собой архитектура нейронных сетей трансформер?
2. Какие основные компоненты включает в себя модель трансформера?
3. Как реализован механизм внимания в архитектуре трансформера?
4. Какие типы входных данных могут быть обработаны с использованием трансформера?
5. Как устроен блок внимания (attention block) в трансформере?
6. Что означает многоголовое внимание (multi-head attention) в трансформере?
7. Какова роль positional encoding в архитектуре трансформера?
8. Как реализован механизм positional encoding в трансформере?
9. Каким образом трансформер обрабатывает переменную длину входных последовательностей?
10. Как трансформеры применяются в задачах обработки естественного языка?
11. Какие преимущества предоставляют трансформеры по сравнению с рекуррентными нейронными сетями?
12. Какие архитектурные изменения были предложены после базовой модели трансформера?
13. Как создать модель нейронной сети трансформера в PyTorch?
14. Как определить архитектуру трансформера в PyTorch?
15. Как добавить блок внимания (attention block) в модель трансформера с использованием PyTorch?
16. Как реализовать механизм внимания (self-attention) в трансформере с помощью PyTorch?
17. Как использовать многоголовое внимание (multi-head attention) в трансформере с использованием PyTorch?
18. Как создать positional encoding для входных данных трансформера в PyTorch?
19. Как обрабатывать последовательности переменной длины в трансформере с использованием PyTorch?
20. Как обучить модель трансформера на многоязычных данных в PyTorch?

СОДЕРЖАНИЕ

Трансформеры	2
Трансформеры	3
1 Общая структуры трансформера	3
2 Scaled Dot-Product Attention (SDPA)	4
Свойства SDPA	5
3 Многоголовое внимание	5
3.1 Нативный блок многоголового внимания	5
3.3 Маскированное многоголовое внимание	7
4 Кодирование поступающей информации	7
4.1 Входные векторы (Input embeddings)	7
4.2 Позиционное кодирование	8
5 Позиционная сеть прямого распространения	8
6 Трасформер-кодер	9
6.1 Трасформер-кодер: архитектура основного блока	9
6.2 Трасформер-кодер: общая архитектура	10
7 Трасформер-декодер	10
7.1 Трасформер-декодер: архитектура основного блока	11
7.2 Трасформер-декодер: общая архитектура	12
8 Режимы использования трансформера	13
8.1 Режим применения (обученного) трансформера (инференс)	13
8.2 Режим обучения трансформера	14
9 Некоторые модификации	15
9.1 Post-LN и Pre-LN	15
9.2 Модель BERT	16
9.3 Модель GPT	17

2. Литература	19
3. Порядок Выполнения лабораторной работы	20
3.1. Исходные данные	20
3.2. Общий план выполнения работы	20
4. Контрольные вопросы	21
Содержание.....	22

Учебное издание

Трансформеры

Методические указания к лабораторной работе № 4

по курсу «Нейронные сети и глубокое обучение»

Составители: Мясников Владислав Валерьевич

Баврина Алина Юрьевна

Самарский национальный исследовательский университет

имени академика С.П. Королева

443086, Самара, Московское шоссе, 34

Отпечатано на кафедре «Геоинформатика и информационная безопасность»

Тираж 20 экз.